Neural ranking

Vladislav Goncharenko

Materials by Kirill Khrylchenko

MIPT,
autumn 2025

Introduction

Where is neural ranking used?

- YouTube, Netflix, TikTok, Spotify
- Amazon, eBay, Etsy
- Twitter/X
- Facebook Ads, Google Ads
- Yandex

Why NNs?

girafe
ai

01

GB vs neural networks

Disadvantages of GB:

- Feature engineering pain: high-cardinality IDs, text/images/graphs
- Poor NN interoperability: no end-to-end learning, no embeddings
- Single-target models: weak at using multiple or rare signals
- Scalability issues: struggles with large data & large models
- Drift handling: must retrain from scratch; no incremental learning

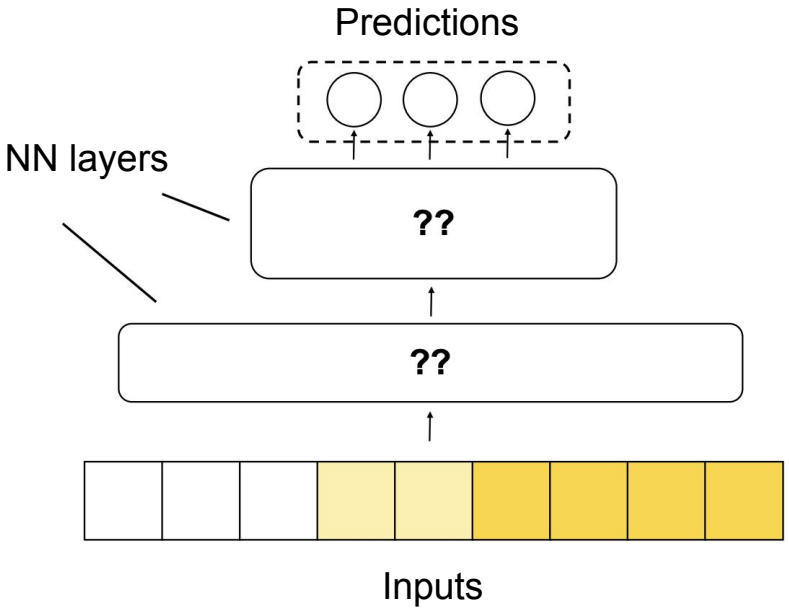
Problems of NN

- Much harder to set up (compared to GB)
- The model may look functional but still fail to deliver profit over boosting
- Many pitfalls: how to encode inputs, which architecture to choose, losses, hyperparameters, optimization
- Requires skills in working with GPUs, distributed training; the runtime requires the ability to deploy neural networks

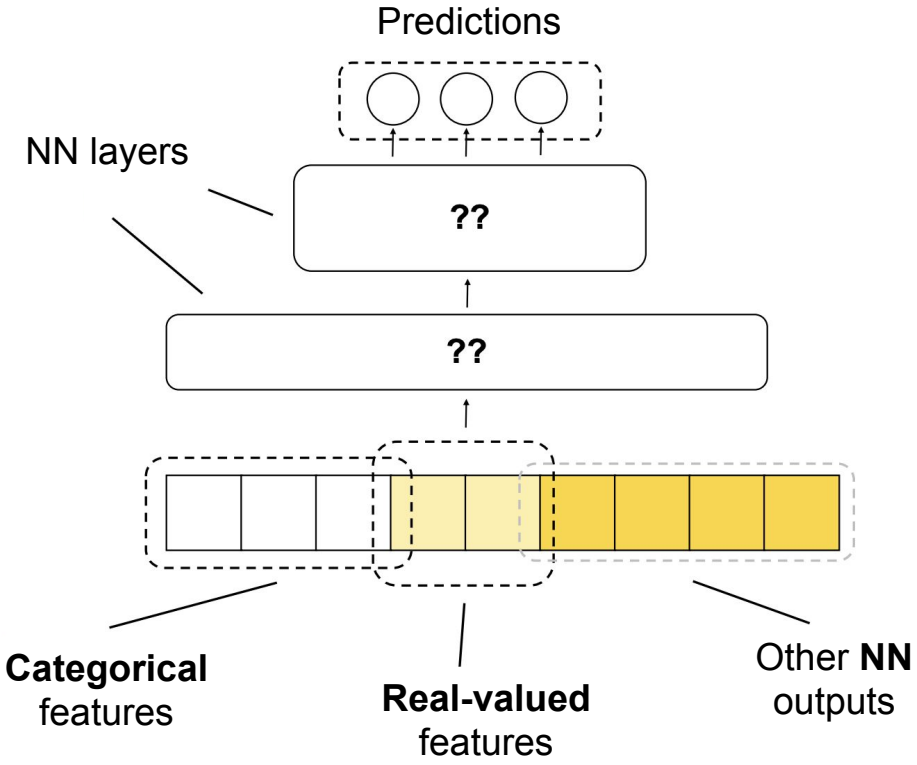
Ranking NN

Starting to build a ranking neural network:

- Form a feature vector
- Apply neural network layers
- Obtain a scalar (or also a vector) at the output



Features encoding



Categorical features

girafe
ai

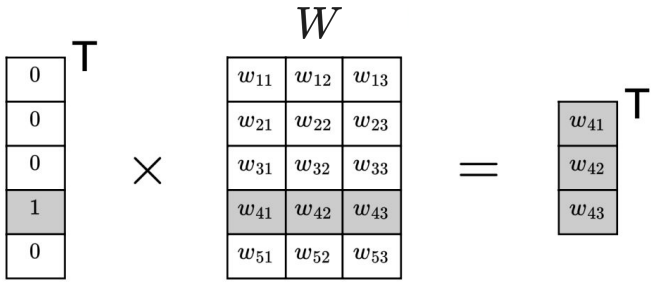
02

Categorical features

- One-hot encoding: $e = (0, \ldots, 1, \ldots, 0) \in \{0, 1\}^n$
- Apply linear layer on it:

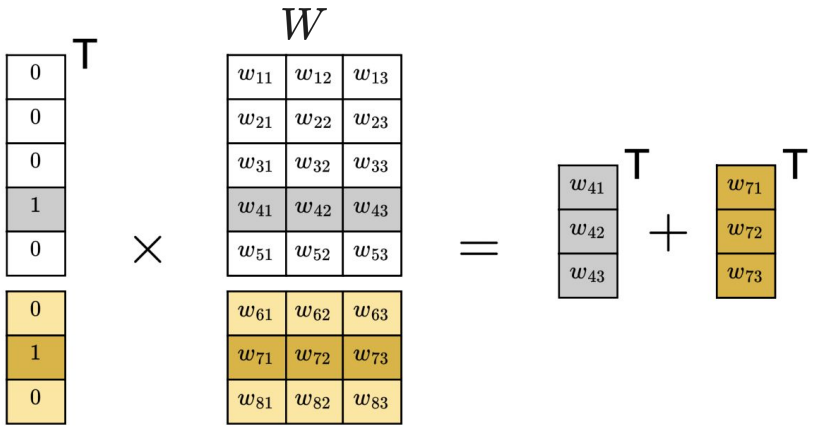
$W \in \mathbb{R}^{\{n \times d\}}$, $e_i W = (W_{ij})_{j=1}^{d}$ - i-th row of W

- W - embedding matrix, each feature -> one embedding
- $e_i W$ - embedding lookup



Categorical features

- Several features: $e \in \{0, 1\}^n$, $f \in \{0, 1\}^m$
- $W \in \mathbb{R}^{\{(n+m)xd\}}$ - concatenation of $W_e \in \mathbb{R}^{\{nxd\}}$, $W_f \in \mathbb{R}^{\{mxd\}}$
- $[e, f]W = eW_e + fW_f$
- Concatenation instead of sum -> different embeddings dim for different features

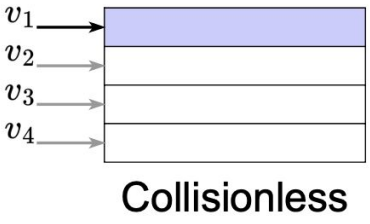


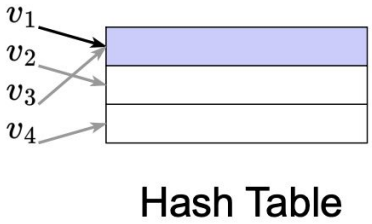
Large embedding matrices

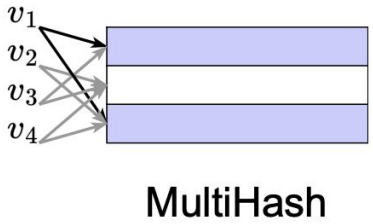
High-cardinality features (e.g., item ID) require a lot of memory.
The total size of embedding tables can reach terabytes: for example, 10 million vectors of dimension 256 (float32) is 9.5 GB.

Possible solutions:

- advanced infrastructure: CPU offloading, sharding (e.g., torchrec)
- hashing trick





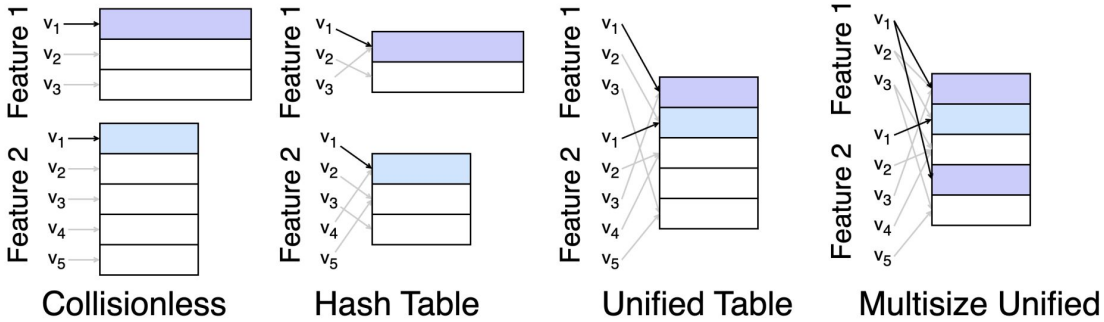


Unified embeddings

A unified embedding matrix for all features:

- To handle collisions, we use multiple hashes
- For different features, we use different hashing seeds
- Item ID: 31415926535 → hashes: [51234, 1839, 22]

  -> embedding: concat[emb1, emb2, emb3]



Embedding dimensions

How to choose embedding dimensionality for different features?

- Use the same dimensionality
- Heuristics: $d = c \log_2 n$, $d = 6\sqrt[4]{n}$ (DCN-v2)
- Regularization

Embeddings in terms of transmitted information:

- Embeddings with d elements, each s bits -> 2**(ds) values
- If all values are equally likely, the entropy of the embedding: $H(v) = ds$
- For n features with p = 1/n:

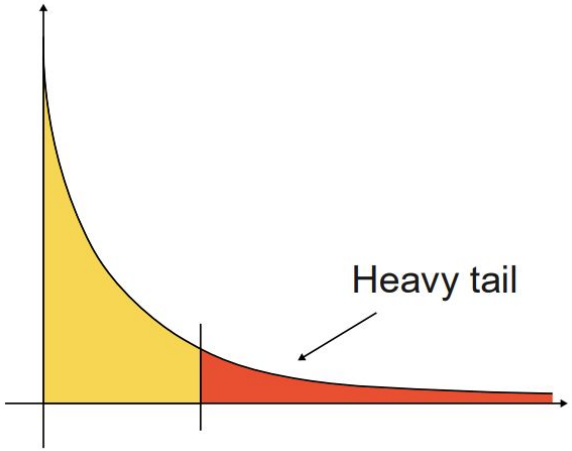
$$H_e = -\sum_{i=1}^{n} p_i \log_2 p_i = \log_2 n$$

Over-training

Embeddings, especially for rare feature values, quickly “memorize” -> overfitting in 1 epoch

Solutions:

- Avoid using IDs :)
- Regularization: L2, dropout, max-norm, frequency clipping
- Hashing trick (implicit regularization)
- Freeze embeddings after the first epoch
- Reset the embedding matrix between epochs



More about embeddings

Batch lookup:

- If many categorical features, performing many small embedding lookups leads to poor GPU utilization due to kernel launch overhead
- You can combine all lookups into one (e.g., EmbeddingCollection in TorchRec)
- Works well with the unified embeddings approach.

Sparse gradients:

- The embedding matrix has very sparse gradients per batch -> no need for full all-reduce in distributed training

Rowwise Adam:

- Two optimizer statistics per embedding row (instead of separate statistics per embedding coordinate)

Real-valued features

girafe ai

03

Real-valued features

Neural networks are sensitive to:

- Outliers: clipping is recommended
- Scale differences: normalization is required
- NaN values: missing values must be handled (imputation, a separate embedding, etc.)

...and even this may not be enough

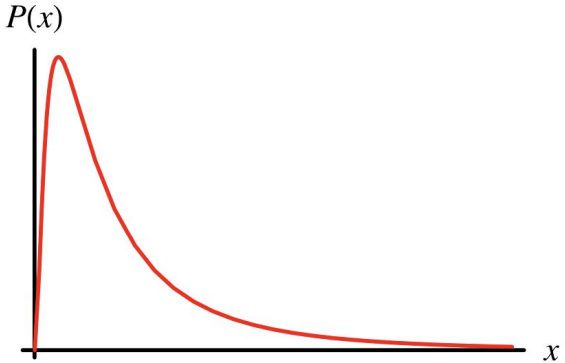
What to do? 🤔

Let's transform them!

Log1p

A simple and popular transformation

- $x \rightarrow \log_2(x) + 1$
- Reduces distribution skewness (e.g., a log-normal distribution becomes normal, and neural networks like normal distributions!)
- “Smooths out” differences between values



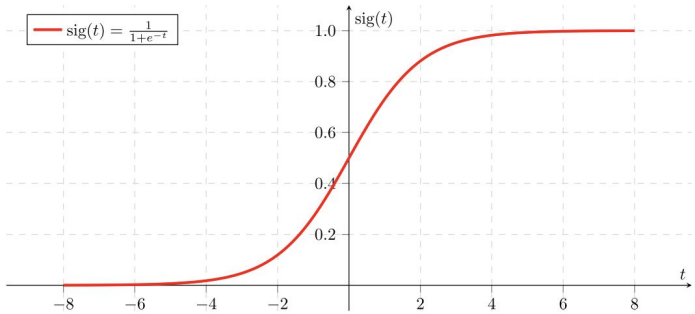
$$P(x) = \frac{1}{x\sigma\sqrt{2\pi}} \exp\left[-\frac{(\ln(x) - \mu)^2}{2\sigma^2}\right]$$

x	1	2	3	10	100	10000
log(x + 1)	0.69	1.1	1.39	2.39	4.61	9.21

Sigmoid Squashing

Alternative to log

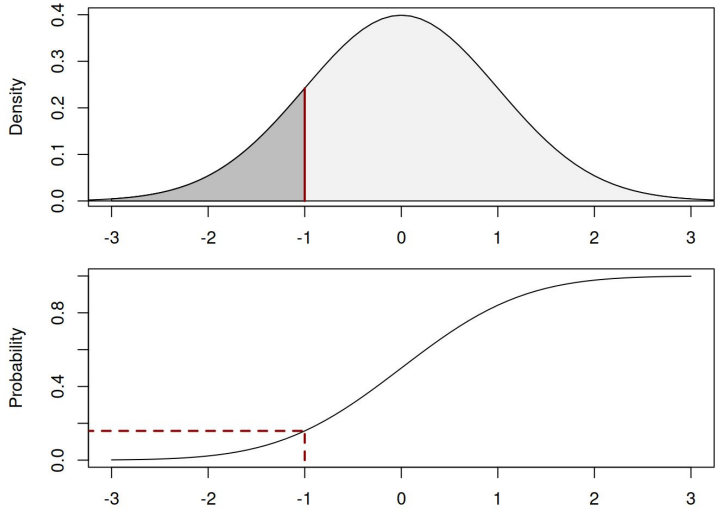
- $x \rightarrow \sigma(ax + b)$
- Strongly smooths out outliers, no clipping required
- Sigmoids were originally commonly used as activation functions
- Important: normalize inputs before sigmoids to avoid vanishing gradients (values hitting the “edges” of the sigmoid)
- You can use multiple sigmoids with different parameters



CDF transformation

Replace a feature value with its distribution estimate

- $x \rightarrow P(X \leq x)$
- Transforms the feature to a uniform distribution on [0, 1]
- Eliminates skewness and outliers



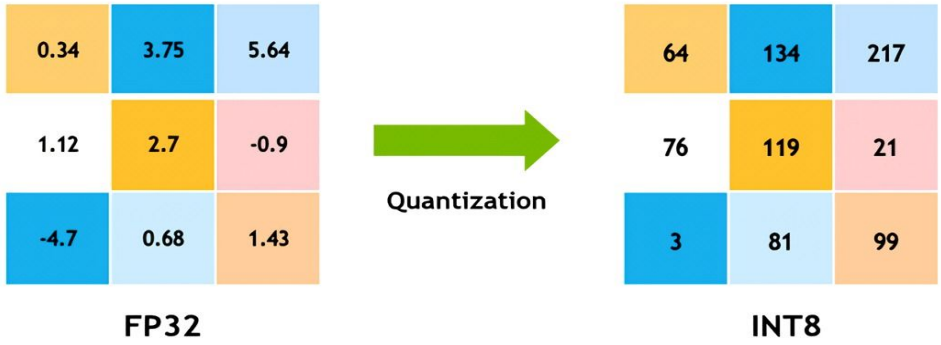
Periodic functions

For expressing periodic dependencies:

- $x \rightarrow [\sin(2\pi c_1 x), \cos(2\pi c_1 x), \ldots, \sin(2\pi c_n x), \cos(2\pi c_n x)]$
- Coefficients can be fixed or learnable
- Used for features related to time
- Similar idea was used for positional embeddings in the original Transformer

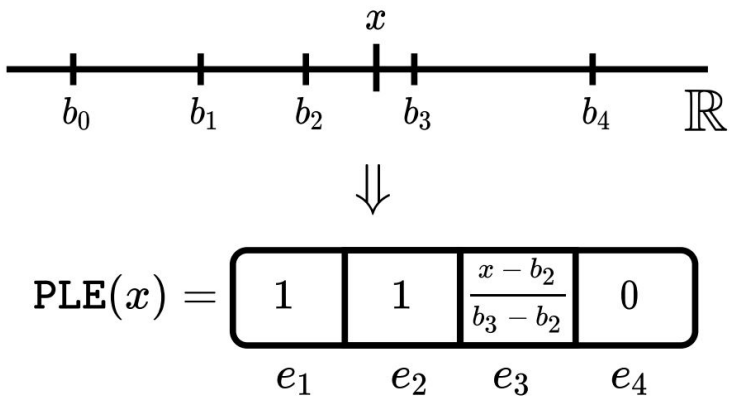
Quantization (Binarization)

- Convert a number into a category:
- Quantization: split values into bins according to quantiles
- Each bin corresponds to a category (bin index)
- Equivalent to piecewise-constant encoding



Piecewise-linear encoding (PLE)

- $x \rightarrow PLE(x)$
- Encode not only the bin index but also the position within the bin
- Preserve the order relationships between bins
- SOTA



Feature interaction layers

girafe
ai

04

Cross-features

- Combinations of input features (the Cartesian product)
- Factorization machines:

$$\hat{y} = w_0 + \sum_{i=1}^{n} w_i x_i + \sum_{i=1}^{n} \sum_{j=i+1}^{n} \langle \mathbf{v_i}, \mathbf{v_j} \rangle x_i x_j$$

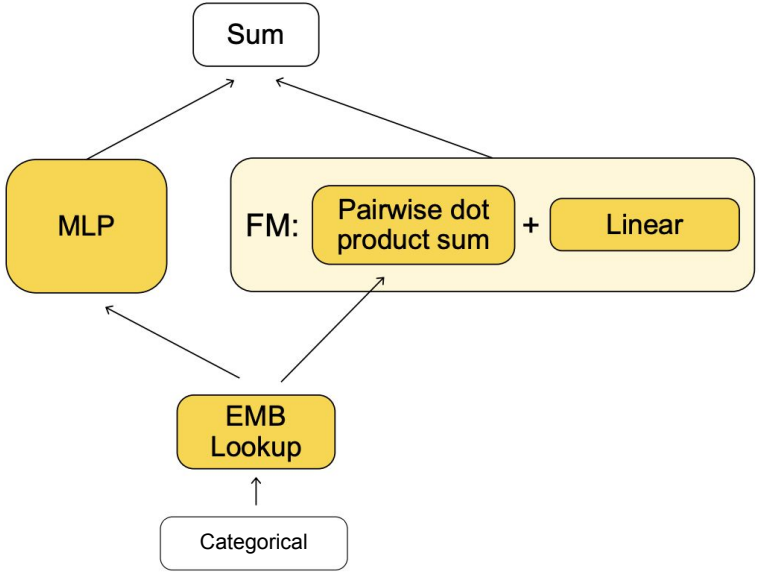
$v_i \in \mathbb{R}^n$ - embedding of i-th feature

$\langle \mathbf{v_i}, \mathbf{v_j} \rangle = \sum_{f=1}^{d} v_{i,f} v_{j,f}$ - dot product of embeddings

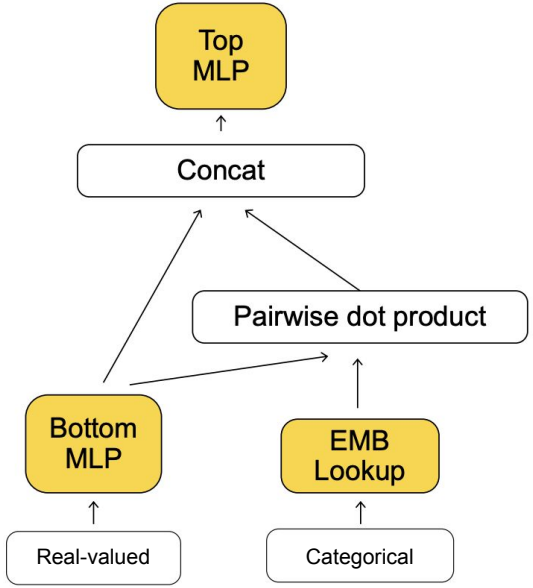
Improvement: create separate embeddings for each feature pair

FM x MLP

DeepFM[1]



DLRM[2]



MLP vs Dot Product

MLP over concatenated vectors cannot learn a dot product!

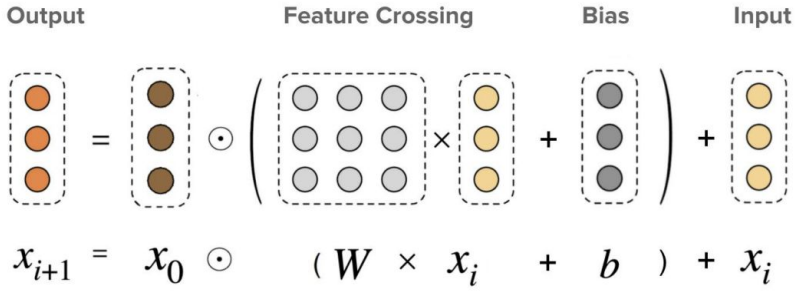
- What to do: multiply vectors element-wise first, then feed into the MLP
- The dot product is then trivially reproduced by a linear layer with all-ones weights
- ...How to generalize this to a large number of vectors?

DCN-v2

- Cross layer:

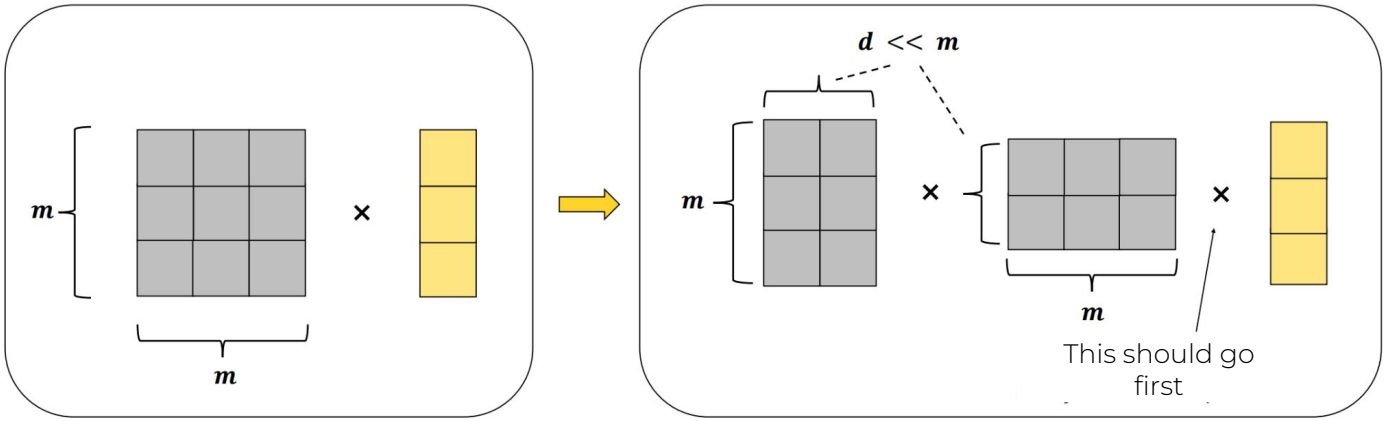
$$x_{l+1} = x_0 \odot (W_l x_l + b_l) + x_l$$

- L layers model all possible feature interactions up to degree L+1
- Diminishing returns: most benefit at 2–3 layers
- **Problem:** very expensive, since it operates on a very wide concatenation of all feature vectors



Low-rank DCN

- Remove bottleneck in multiplying by a large matrix using low-rank decomposition



DCN-v2

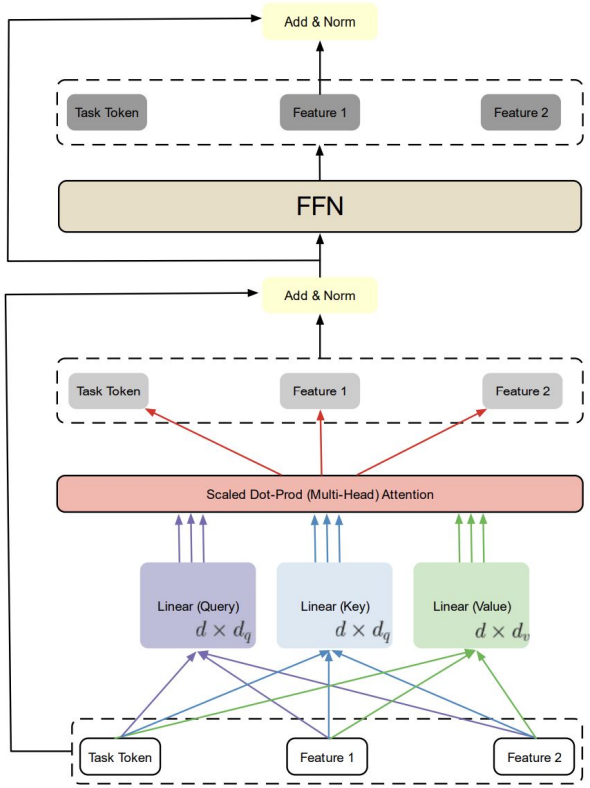
- Explicitly models low-order feature interactions
- Effective only on raw embeddings, not on abstract vectors (e.g., Transformer outputs)
- Combine with MLP:
  - MLP implicitly models high-order feature interactions
  - In practice: apply DCN first, then MLP (parallel application works worse)
  - Usually, the DCN output vector is narrowed before feeding into MLP

Attention

Intuition: Attention mechanisms should model feature interactions effectively.

AutoInt:

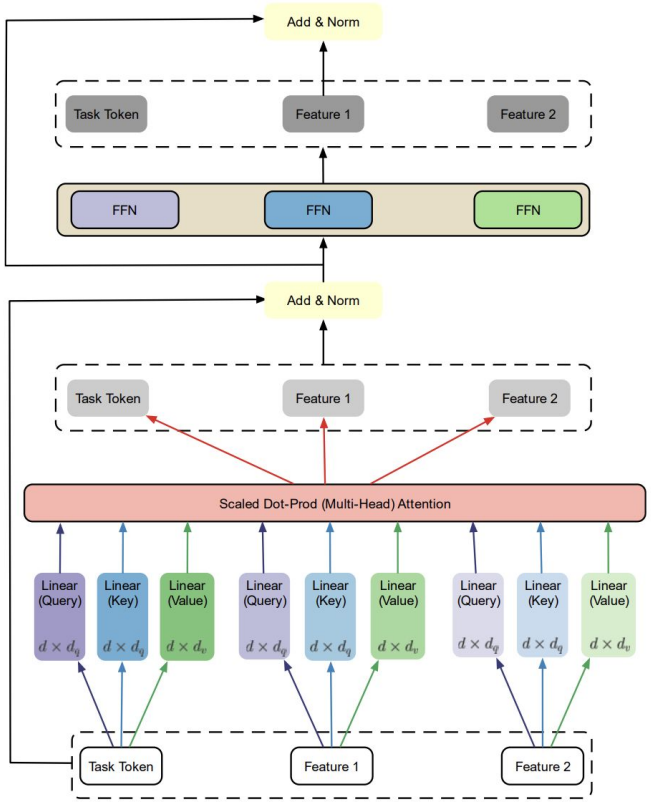
- Obtain feature embeddings of the same dimension (for real-valued features, multiply the scalar by a learnable vector)
- Apply multi-head attention
- Concatenate all resulting vectors and feed into an MLP



Attention

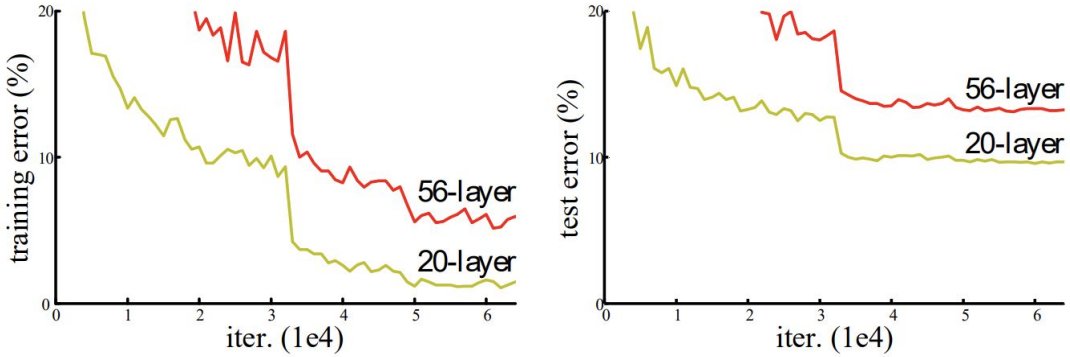
Hiformer:

- Features are heterogeneous with different structures
- Standard attention applies the same Q, K, V matrices to all features
- Different feature pairs need to be processed differently (field-awareness!)
- Solution: heterogeneous attention – separate Q, K, V matrices for each feature



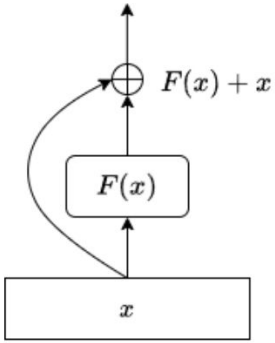
What’s wrong with MLP?

- MLP: fully connected neural network with linear layers and nonlinearities in between (usually ReLU)
- Issues:
  - Harder to optimize as the number of layers increases
  - Vanishing gradients (normalization helps)



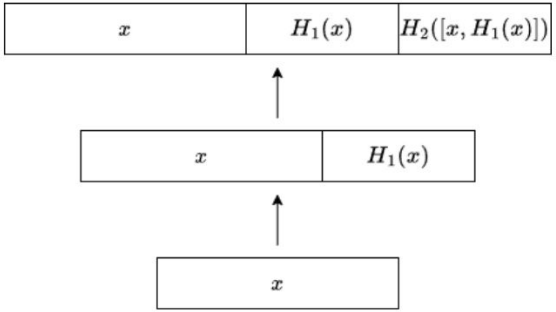
MLP alternatives

ResNet



$$\text{Output} = x + F(x)$$

DenseNet



$$\text{Output} = \text{concat}\big(x, F(x)\big)$$

Multi-task

girafe ai

05

Multi-task

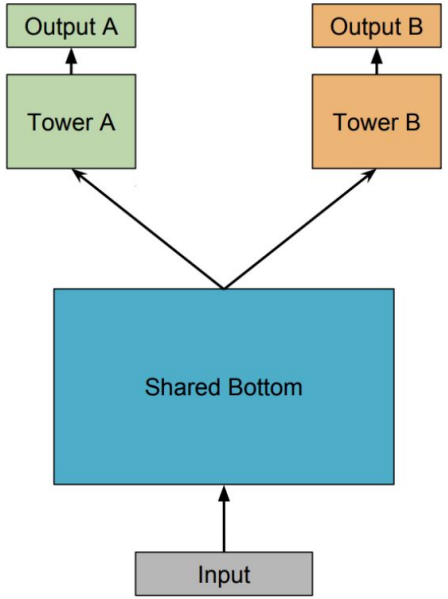
- Many signals in RecSys (likes, dislikes, views, shares, etc.)
- How to take into account?
  - each model for each signal
  - make new target (like = 2, dislike = -3, view = 1, share = 3, ...)
  - weighted sampling according to importance of signal
- Problems:
  - Expensive to maintain; some signals cannot be learned in isolation
  - Pareto front of prediction quality shifts with every significant change in ranking -> need to re-tune targets/weights

Multi-task learning

- Related signals can help each other:

  P(order | view) = P(order | click) * P(click | view)

- Neural networks enable multi-head models:
  - Shared backbone: common layers for all tasks
  - Task-specific heads: separate layers for each task
  - Extremes: Fully separate models: all layers task-specific
  - Fully shared model: backbone only, task-specific layers minimal
- Advantages:
  - One model for all signals -> saves resources
  - Knowledge transfer: improves overall performance



Negative Transfer

- During training, different tasks can pull the model in different directions (gradients for different tasks are not aligned)
- Task conflict: performance on one task worsens in favor of another
- Solutions:
  - Increase model capacity: larger models can better handle conflicts
  - Add more task-specific layers
  - Mixture of Experts: selectively route tasks to different experts

Mixture of Experts

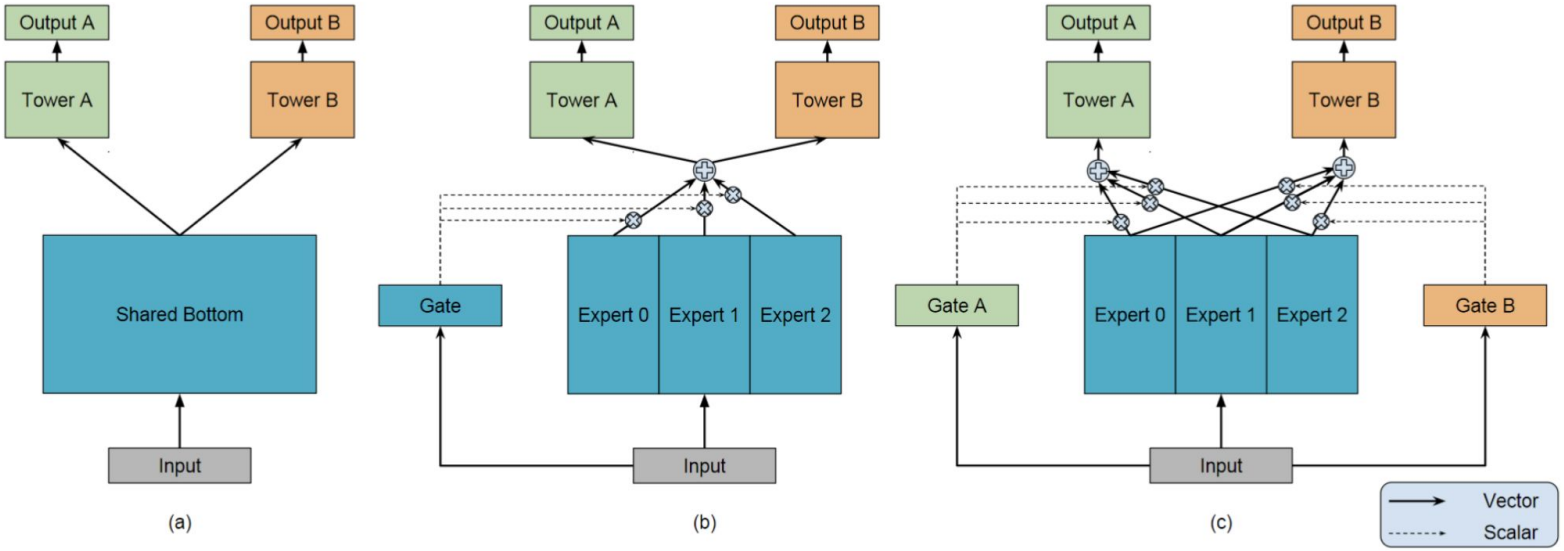
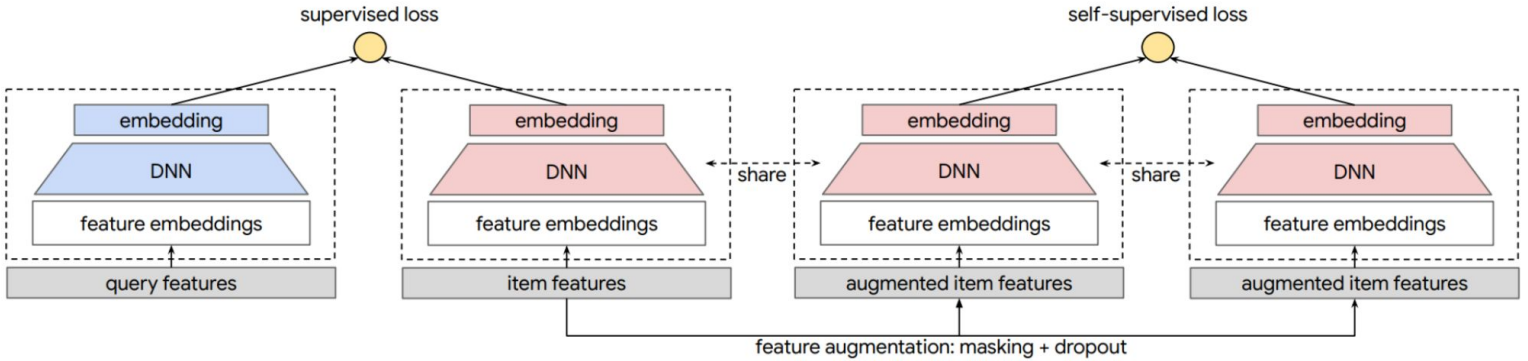


Figure 1: (a) Shared-Bottom model. (b) One-gate MoE model. (c) Multi-gate MoE model.

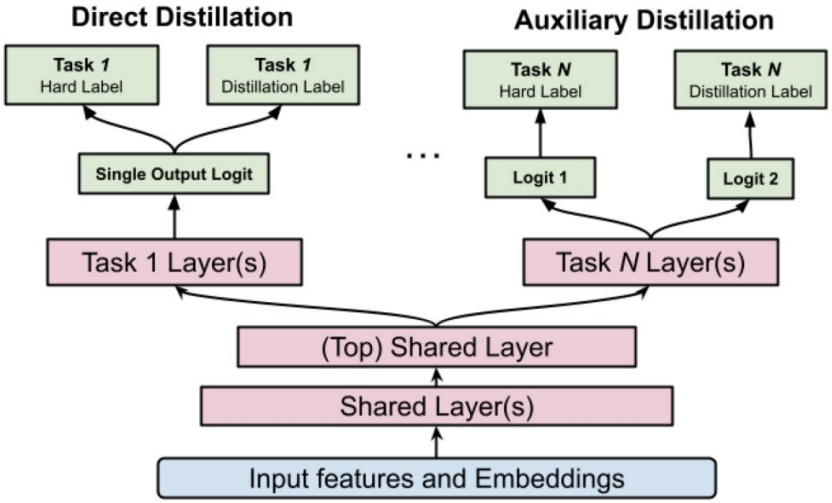
Self-supervised Learning

- Idea: “corrupt” features at model input as a form of augmentation
- Benefits:
  - Improves performance on long-tail / rare features
  - Enhances generalization through regularization

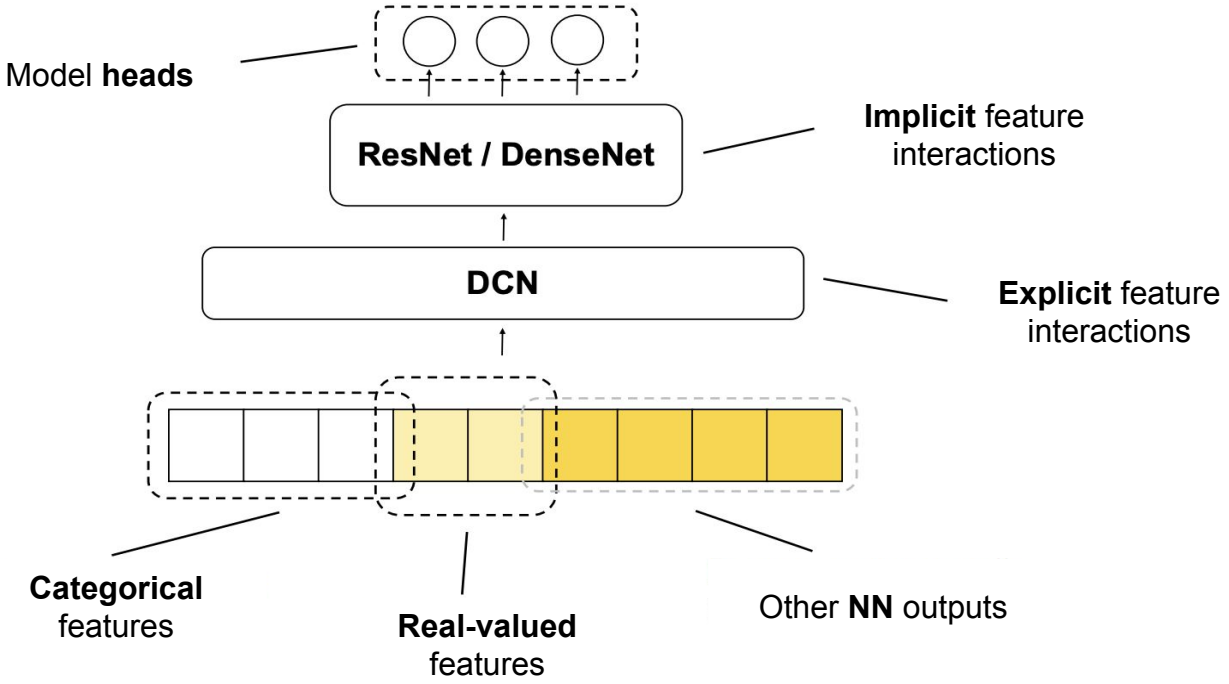


Knowledge Distillation

- Train a teacher (large model configuration)
- Train a student on a combination of true labels and teacher's predictions
- Auxiliary distillation: train a separate student head on teacher's predictions



Recap


Model **heads**
ResNet / DenseNet
Implicit feature interactions
DCN
Explicit feature interactions
Categorical features
Real-valued features
Other **NN** outputs

Read more

- Unified embeddings: https://arxiv.org/pdf/2305.12102
- DeepFM: https://arxiv.org/abs/1703.04247
- DLRM: https://arxiv.org/abs/1906.00091
- DCN-v2: https://arxiv.org/pdf/2008.13535
- AutoInt: https://arxiv.org/abs/1810.11921
- HiFormer: https://arxiv.org/pdf/2311.05884
- MoE in RecSys: https://dl.acm.org/doi/pdf/10.1145/3219819.3220007
- Self-supervised learning: https://arxiv.org/pdf/2007.12865
- Knowledge distillation: https://arxiv.org/abs/2408.14678

Thanks for attention!

Questions?

---

girafe
ai